

EduProva — Database Architecture

Knowledge Transfer (KT) Notes: MongoDB vs PostgreSQL vs RDS vs Aurora

Session Goal

This document summarizes the database architecture discussion for EduProva, prepared for tomorrow's KT session. The core question addressed: should EduProva use MongoDB only, PostgreSQL only, or a combination of both — with the primary business driver being **cost reduction**.

Key Context

- MongoDB is **not yet integrated** into the EduProva project — this is a from-scratch decision, not a migration.
- The application has 16 core data categories: Users, Courses, Enrollments, Orders, Payments, Subscriptions, Invoices, Contracts, Chat, Messages, Notifications, Content, Reviews, Activity, Listings, Dynamic Metadata.
- Primary business goal: reduce infrastructure and operational cost.

Bottom-Line Recommendation

Start EduProva with a single relational database (PostgreSQL) as the primary store, using JSONB for flexible fields. Introduce MongoDB later, only for specific workloads (chat, activity streams, highly dynamic content) once real usage data justifies it.

1. Can All 16 Categories Go Into MongoDB?

Yes, technically possible. MongoDB can handle payments, subscriptions, users, and orders. "MongoDB can't run this" is not the correct framing. The real decision is not about raw capability — it is about **data consistency, relationships, transactions, reporting/querying, and operational cost.**

1.1 Latency

Latency is **not an inherent MongoDB problem**. A properly designed MongoDB cluster in the same AWS region, with correct indexes, connection pooling, and a good shard key, delivers low latency. Latency issues typically come from architecture and network design — for example, if MongoDB Atlas sits in a separate AWS account/environment from EKS without proper private connectivity.

Conclusion: Latency = an architecture/network/query design problem, not a MongoDB limitation.

1.2 Payments & Subscriptions in MongoDB

The biggest concern raised was multi-collection consistency — e.g., a single \$999 payment needs to update Payments, Orders, Subscriptions, Enrollments, and Invoices together. MongoDB **does support ACID multi-document transactions**, so this workflow can technically be made consistent in MongoDB as well. Saying "MongoDB can't guarantee payment consistency" is incorrect.

1.3 Where PostgreSQL Has a Natural Advantage

- **Relationships:** User → Order → Payment → Subscription → Enrollment → Course maps naturally to relational tables with foreign keys and SQL joins. In MongoDB, these relationships must be modeled manually at the application/data-model level.
- **Reporting:** Queries like "revenue per course over the last 6 months" are simple SQL joins + GROUP BY in PostgreSQL. The same logic in MongoDB requires aggregation pipelines that get complex as relationships grow.
- **Data duplication risk:** MongoDB read-optimization often means denormalizing data (e.g., embedding user name inside an order document). If a user's name changes, old embedded copies across many documents need a careful update strategy.

2. MongoDB-Only Suitability by Module

■ does not mean impossible — it means it requires careful schema and transaction design.

Area	MongoDB-Only Fit
Users	■ Good
Courses	■ Good
Enrollments	■ Relationships-heavy
Orders	■ Transactional
Payments	■ Strong consistency needed
Subscriptions	■ Transactional
Invoices	■ Financial data
Contracts	■ Relationships-heavy
Chat	■ Excellent fit
Messages	■ Excellent fit
Notifications	■ Good
Content	■ Excellent fit
Reviews	■ Good
Activity	■ Excellent fit
Listings	■ Good
Dynamic Metadata	■ Excellent fit

Reliable takeaway: Payments, Orders, Subscriptions, Invoices, Contracts and Enrollments are the modules where MongoDB requires the most careful design; Chat, Messages, Activity, Content, Listings and Dynamic Metadata are naturally excellent fits for MongoDB's document model.

3. Cost Perspective — One DB vs Two DBs

Option 1: Two Databases (PostgreSQL + MongoDB)	Option 2: MongoDB Only
PostgreSQL infra + MongoDB infra, each with its own monitoring, backups, security, and maintenance. Higher engineering complexity.	Single database platform → simpler operations. But a production MongoDB cluster (replicas, storage, backup, scaling) still costs money — "MongoDB-only = definitely cheaper" is not guaranteed.

At low initial traffic (e.g., ~1,000 users, low payments/chat/activity volume), running two databases (Aurora/RDS + MongoDB Atlas, each with monitoring, backups, network, and security) is unnecessary cost and complexity. A single PostgreSQL database with JSONB for flexible fields is a much cleaner starting point.

4. Final Architecture Decision — Phased Approach

Phase 1 — Start with PostgreSQL Only

EKS → RDS/Aurora PostgreSQL, covering all core modules from day one:

- Users, Courses, Enrollments, Orders, Payments, Subscriptions, Invoices, Contracts
- Flexible/semi-structured fields handled using **PostgreSQL JSONB** (covers most use cases that would otherwise push toward MongoDB)

Result: MongoDB cost = 0 at launch; one less database to maintain.

Phase 2 — Add MongoDB Only When Justified

Introduce MongoDB Atlas only when a specific workload proves the need — e.g., chat reaching millions of messages, high-volume activity/event streams, or highly dynamic content structures. MongoDB should be added *because a workload demands it*, not because "MongoDB is good."

Phase 3 — Optimize Using Real Metrics

Use CloudWatch metrics (CPU, connections, latency, IOPS, storage, query performance) to decide which specific workloads should move to MongoDB. Example outcome: Chat → MongoDB is clearly better; Payments → PostgreSQL is clearly better; Course Content → JSONB is often sufficient.

Module Assignment Summary

PostgreSQL (from Day 1)	MongoDB (added later, if justified)
Users, Courses, Enrollments, Orders, Payments, Subscriptions, Invoices, Contracts	Chat, Messages, Activity, Dynamic Content, Reviews, Listings, Dynamic Metadata, Notification history

Note: Notification history storage is a PostgreSQL/MongoDB question, but the actual delivery mechanism usually depends on a separate queue/event + notification service design.

Decision Flow

- Not "MongoDB first" — Not "PostgreSQL first" — Not "both, because it looks impressive"
- Start from Requirements → PostgreSQL → JSONB for flexible data → Measure real workload → Add MongoDB only if justified

5. RDS PostgreSQL vs Aurora PostgreSQL

Given the cost-reduction goal, Amazon RDS for PostgreSQL was evaluated as an alternative to Aurora PostgreSQL. Both use the same PostgreSQL engine, so SQL, tables, joins, transactions, foreign keys, JSONB, and application-level CRUD behave the same way — including multi-step transactional flows like Payment → Subscription → Enrollment updates in a single BEGIN...COMMIT block.

Feature	RDS PostgreSQL	Aurora PostgreSQL
PostgreSQL engine / SQL / Transactions / Joins / JSONB	■	■
Automated backups	■	■
Read replicas	■	■
Multi-AZ high availability	■	■
Serverless option	Limited / different options	■ Aurora Serverless v2
Aurora-specific distributed storage architecture	■	■
Typical starting cost	Usually simpler / lower	Can be higher, workload-dependent

Recommendation for EduProva

Given: cost sensitivity + from-scratch build + MongoDB not yet integrated + relational core data → **start with RDS PostgreSQL**, not Aurora.

- Aurora is mainly attractive for high-scale, high-availability workloads (its own distributed storage, writer + multiple readers). RDS also supports HA and read replicas, but with a different architecture.
- Start: RDS PostgreSQL → Users, Courses, Enrollments, Orders, Payments, Subscriptions, Invoices, Contracts, with JSONB for flexible fields.
- Migration path: if metrics later show the workload genuinely needs Aurora's capabilities, RDS → Aurora migration can be planned at that point.

Caveat: "RDS is cheaper than Aurora" should not be assumed blindly — actual billing depends on instance size, Multi-AZ configuration, storage type, IOPS, backups, read replicas, and real traffic. Compare based on EduProva's expected traffic and HA requirements, not on engine name alone.

6. One-Line Summary for the Presentation

“To minimize both infrastructure and operational cost, we will start EduProva with RDS PostgreSQL as the primary database and use JSONB for flexible data. We will introduce MongoDB only when a specific workload — such as high-volume chat, activity streams, or highly dynamic documents — demonstrates a clear technical benefit.”

Key Talking Points for KT

- MongoDB can technically handle all 16 categories, including payments — but that's not the right test.
- The real decision factors are: relationships, transactions, reporting complexity, and operational cost.
- Latency is an architecture/network concern, not an inherent MongoDB weakness.
- Start lean: PostgreSQL (RDS) + JSONB from day one; add MongoDB later only where metrics justify it.
- RDS PostgreSQL is the recommended starting engine over Aurora, given the cost-first, from-scratch context.